VarTable

VarTable est un paquet pour rendre la réalisation
de tableaux de signes et de variations plus simple
Ce paquet est construit sur Cetz (version : 0.2.1)

Table des matières

1 - Introduction 3
2 - Description générale 4
tabvar 4
Parameters 4
variable 4
label 4
domain 5
contents 5
table-style 5
nocadre 5
arrow-mark 5
arrow-style 5
line-0 6
undefined-bar-alt 6
line-style 6
hatching-style 6
first-column-width 6
first-line-height 6
element-distance 7
values 7
fill-color 7
add 7
3 - Paramètre de contenu 8
3.1 - Format pour les signes 8
3.1.1 - Un array classique pour les signes 8
3.1.2 - Barre de séparation customisée 9
3.1.2.1 - Style de barre 9
3.1.2.2 - Type de barre 9
3.1.3 - Ignorer une valeur 11
3.1.4 - Hachurage pour un intervalle non défini 12
3.2 - Format pour les variations 14
3.2.1 - Array classique pour les sous-tableaux de variations 14
3.2.2 - Valeurs interdites 14
3.2.3 - Ignorer une valeur 16
3.3 - Hachurage pour un intervalle non défini 16
4 - Redimensionner le tableau 19
4.1 - Première ligne et première colonne 19
4.2 - Redimensionner l'espacement entre les éléments 19
4.3 - Redimensionner la hauteur des sous-tableaux 20
5 - Customisation du hachurage 20
6 - Ajouter des valeurs dans les sous-tableaux de variations 24

7 - Ajouter ce que vous voulez avec `add` 26

1 - Introduction

Ce paquet a été réalisé pour rendre la création de tableau de signes et de variations plus simple. Pour cela, ce paquet fournit la fonction « `tabvar` », dont les paramètres sont décrit dans cette documentation.

Si vous rencontrez un bug, merci de me prévenir via mon GitHub.

P-S. Je sais que mon français n'est pas des plus excellents, donc si cette documentation vous brûle les yeux et que vous avez un peu de temps à perdre, vous serez les bienvenus pour améliorer cette documentation.

Remerciements : Je tiens à remercier supersurviveur et dododu74, pour leur aide au début du projet (notament la correction des premières documentations), ainsi que Akilon27, sans qui les tableaux ne seraient pas aussi customisables.

2 - Description générale

tabvar

Retourne un tableau de variation

Parameters

```
tabvar(
  variable: content,
  label: array,
  domain: array,
  contents: array,
  table-style: style,
  nocadre: bool,
  arrow-mark: mark,
  arrow-style: style,
  line-0: bool,
  undefined-bar-alt,
  line-style: style,
  hatching-style: tiling,
  first-column-width: length,
  first-line-height: length,
  element-distance: length,
  values: array,
  fill-color: color,
  add: content
)
```

variable `content`

`variable` est la variable qui contient la variable du tableau (comme x ou t)
Exemple : si la variable de la fonction est t, alors :

```
variable : $ t $
```

Default: `$ x $`

label `array`

`label` est un array qui contient des array de longueur 2, une pour chaque ligne du tableau, dont le premier élément est le titre de la ligne et le second est le type de la ligne : signes (« s ») ou variations (« v »)
Exemple : pour le tableau de variations de la fonction f, vous devriez écrire :

```
label : (
  ([Signe de $f$], "s"), // la première ligne est un tableau de signe
  ([Variations de $f$], "v") // la seconde ligne est un tableau de variations
)
```

Default: `()`

domain `array`

Les valeurs prises par la variable.
Par exemple, si votre fonction change de signe ou atteint un extrémum pour $x \in \{0; 1; 2; 3\}$, vous devriez écrire :

```
domain: ($0$, $1$, $2$, $3$)
```

Default: `()`

contents `array`

the content of the table
see below for more details Le contenu du tableau
Voir 2.2 pour plus de détails

Default: `((),)`

table-style `style`

Optionelle
Le style du tableau
Le type style est défini par Cetz, aussi je vous recommande de vous référer au manuel de Cetz.
Attention : Si vous ne mettez pas le paramètre de style : `mark` à `none`, alors toutes les lignes du tableau auront une tête en flèche

Default: `(stroke: 1pt + black, mark: (symbol: none))`

nocadre `bool`

Pour cacher le cadre externe du tableau

Default: `false`

arrow-mark `mark`

Le style de la tête de la flèche.
NB. le type `mark` est défini par Cetz

Default: `(end: "straight")`

arrow-style `style`

Optionelle :
Le style des flèches.
Attention : le paramètre `mark` est supplanté par le paramètre `arrow-mark`

Default: `(stroke: black + 1pt)`

line-0 `bool`

Optionelle
Si vous voulez remplacer la barre par défaut dans les tableaux de signes, par une barre avec un 0 en son centre

Default: `false`

undefined-bar-alt

Optionelle
change les bars d'indéfinies au début et à la fin des sous-tableaux pour copier le style de tkz-tab

Default: `false`

line-style `style`

Optionelle
Si vous voulez le style de toutes les barres de séparation entre les signes

Default: `(stroke: black + 1pt)`

hatching-style `tiling`

Optionelle
Le style des hachures, s'il y a des zones hachurées

Default: `tiling(size: (30pt, 30pt))[`

```
    #place(line(start: (0%, 100%), end: (100%, 0%), stroke: 2pt))
    #place(line(start: (-100%, 100%), end: (100%, -100%), stroke: 2pt))
    #place(line(start: (0%, 200%), end: (200%, 0%), stroke: 2pt))
  ]
```

first-column-width `length`

Optionelle
Change la largeur de la première colonne

Default: `none`

first-line-height `length`

Change la hauteur de la première ligne (celle du domaine et de la variable)

Default: `none`

element-distance `length`

Optionelle
Change la distance entre deux éléments

Default: `none`

values `array`

Pour ajouter des valeurs entre deux valeurs prédéfinies

Default: `((),)`

fill-color `color`

Optionelle
La couleur de remplissage pour les valeurs sur les flèches (devrait être identique à l'arrière-plan du tableau) Par défaut : white

Default: `white`

add `content`

Pour ajouter davantage d'éléments, via Cetz

Default: `()`

3 - Paramètre de contenu

Le paramètre contenu est un array avec un élément par ligne (par label).

Les éléments sont eux-mêmes chacun des arrays avec un élément pour chaque colonne, avec un format différent pour les signes et les variations, qui seront détaillés ci-dessous.

3.1 - Format pour les signes

Il doit être positioné au même index dans l'array `contents` qu'un label possèdant le string `"s"`, ce qui indique que la ligne doit être considérée comme un tableau de signes.

De plus, il doit contenir autant d'éléments moins 1 que le *domain* (un par intervalle), plus un argument optionel si le dernier élément est non défini.

Chaque élément doit être d'une de ces formes :

- `()` - Vide : pour étendre le dernier signe en partant de la gauche sur les intervalles marqués vides
- `body` - Le cas basique, constitué du type body de Typst, comme `$ + $` ou `$ - $`
- `(style de la barre, body)` - Pour appliquer un style particulier à la barre de **devant** le signe. Ce style peut être : `"|"` barre simple, `"||"` double barre, ou `"0"` pour une barre avec un 0 en son centre. Ce peut aussi être un contenu quelconque (`[…]`, `$…$`, `box(…)`) : il est posé au centre de la barre, comme le 0 — par exemple une case vide à compléter, `(box(width: 0.8em, height: 0.8em, fill: white, stroke: (dash: "dotted")), $-$)`, attention le tableau ne s'adapteras pas à cette injection.

NB : le paramètre `line-0` change la barre par défaut pour la barre avec un zéro `"0"`.

Vous pouvez mettre en plus à la fin la chaine de caractères `"||"`, pour ajouter une double barre à la toute fin.

3.1.1 - Un array classique pour les signes

Un tableau de signe classique :

```
#tabvar(
  init: (
    variable: $t$,
    label: (([signe], "s"),),
  ),
  domain: ($2$, $4$, $6$, $8$),
  contents: (($+$, $-$, $ + $),),
)
```

t	2	4	6	8
signe	+	−	+	

Un exemple plus complexe :

```
#tabvar(
  variable: $t$,
  label: (
    ([signe 1], "s"),
    ([signe 2], "s"),
  ),
  domain: ($ 2 $, $4$, $6$, $8$),
  contents: (
    ("Hello world !", $-$, $ 3 / 2 $),
    ($1/3/9/27$, $+$, "Goodbye word !"),
  ),
)
```

t	2	4	6	8
signe 1	Hello world !	$-$	$\frac{3}{2}$	
signe 2	$\frac{\frac{1}{3}}{\frac{9}{27}}$	$+$	Goodbye word !	

Note. Sur le second example, le tableau est comprimé à l'aide de la fonction scale

3.1.2 - Barre de séparation customisée

3.1.2.1 - Style de barre

Vous pouvez modifier le style de la barre.

Le style de la barre est un dictionary, de type `"style"` défini par Cetz.
Pour faire simple, si vous voulez changer uniquement le stroke des barres, vous n'avez qu'à mettre `(stroke: votre stroke)`.
Pour des usages plus complexes, référez-vous au manuel de Cetz.

Exemple.

```
#tabvar(
  line-style: (
    stroke: (paint: red, dash: "dashed")
  ),
  variable: $t$,
  label: (([signe], "s"),),
  domain: ($2$, $4$, $6$,),
  contents: (
    ($+$, $-$),
  ),
)
```

t	2	4	6
signe	$+$		$-$

3.1.2.2 - Type de barre

Pour tous les signes sauf le premier, au lieu de placer directement un signe, vous pouvez mettre un couple, dont le premier élément définit le type de la barre placée avant le signe.
Il y a trois types différents de barres :

- `"|"` : une barre simple
- `0` : une barre avec un 0 en son centre
- `||` : une double barre pour les valeurs interdites

Exemple.

```
#tabvar(
  variable: $t$,
  label: (([signe], "s"),),
  domain: ($2$, $4$, $6$, $8$, $10$),
  contents: (
    (
      $+$,
      ("|", $-$),
      ("0", $-$),
      ("||", $+$)
    ),
  ),
)
```

t	2	4	6	8	10
signe	$+$	$-$	0	$-$	$+$

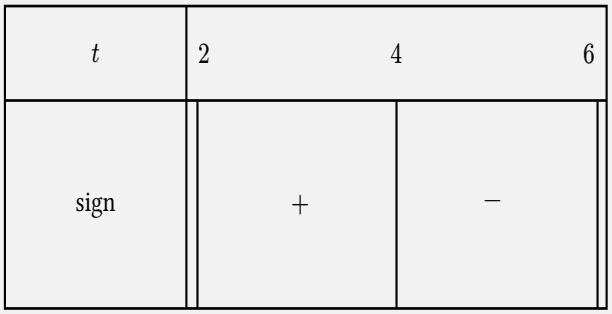
Si vous voulez avoir une double barre avant le premier signe, vous pouvez utiliser le couple avec en premier élément `"||"`, à la place du premier signe ; pour mettre une double barre à la fin, ajoutez à la fin de l'array la chaine de caractères `"||"`, toute les autres types de bar sont également supporté, notament l'injection d'un contenus quelconque au milieu de la bar.

Example :

```
#tabvar(

  variable: $t$,
  label: (([signe], "s"),),

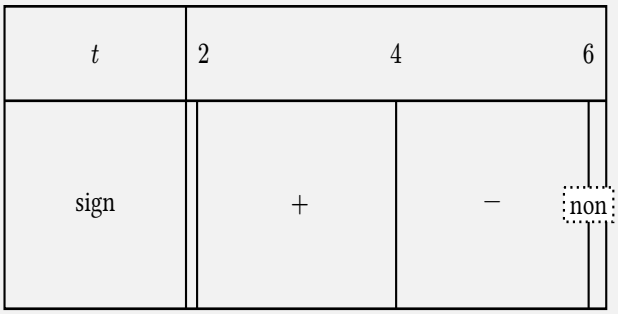
  domain: ($2$, $4$, $6$),
  contents: (
    (
      ("||", $+$),
      $-$,
      "||"
    ),
  ),
)
```




```
#tabvar(

  variable: $t$,
  label: (([signe], "s"),),

  domain: ($2$, $4$, $6$),
  contents: (
    (
      ("||", $+$),
      $-$,
      box(
        fill: white,
        stroke: (dash: "dotted"),
        inset: 1mm
        )[
          non
        ]
    ),
  ),
)
```



3.1.3 - Ignorer une valeur

Quand votre tableau de signes possède plus d'un sous-tableau, alors vous seriez tenté de vouloir mettre un même signe pour plusieurs valeurs du domaine.
Pour cela, c'est assez simple : au lieu de mettre un signe directement, mettez simplement un couple vide `()`

Example :

```
#tabvar(
  line-0: true,
  variable: $t$,
  label: (
    ([signe], "s"),
  ),
  domain: ($2$, $4$, $6$, $8$),
  contents: (
    ($+$, (), $-$),
  ),
)
```

t	2	4	6	8
signe		+	0	−

3.1.4 - Hachurage pour un intervalle non défini

Il se peut que vos fonctions ne soient pas définies sur un ou plusieurs intervalles malheuresement présent(s) dans le domaine du tableau de signes. Pour cela, la convention veut que l'on hachure la zone en question.
Étant donné que les signes portent sur les intervalles du domaine, il en résulte une syntaxe ralativement simple d'usage, dont on poura distinguer quatre cas :

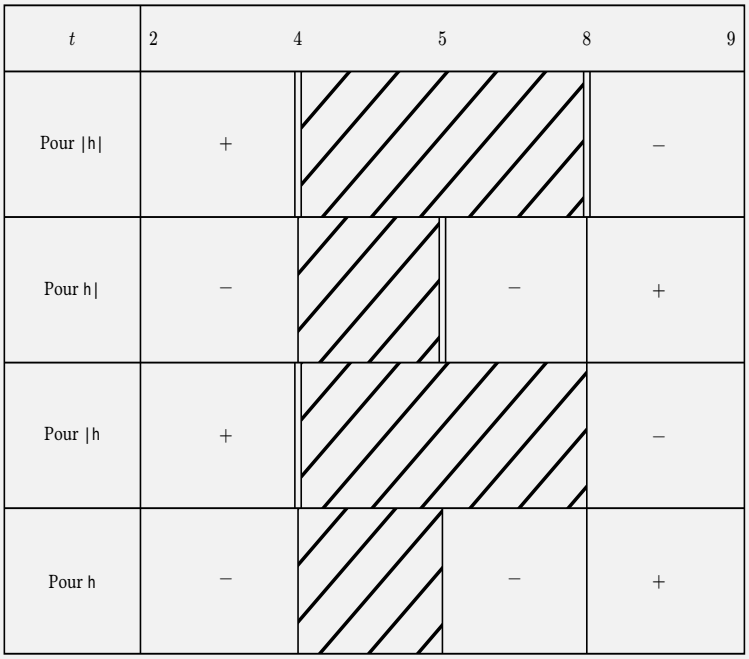
- le premier cas et le plus courant, celui où les deux bornes de l'intervalle non défini le sont également, ainsi à la place où vous auriez mis votre signe (ou tout autre élément), vous renseignerez l'élément suivant : `"|h|"`
- le deuxième cas, relativement présent également, est celui où les deux bornes sont définies, ainsi vous omettrez les deux barres « | » de l'élément présenté ci-dessus, ie. vous renseignerez `"h"`
- les deux autres cas, sont celui où une seule des deux bornes est définie. Ainsi, comme vous l'aurez sans doute compris, retirej (resp. ajoutez) la barre pour le côté où l'élément est défini (resp. non défini). Soit : pour une valeur définie à gauche `"h|"` ; pour une valeur définie à droite `"|h"`

Remarque. Vous avez sans doute compris que la bare « | » symbolise les doubles barres de valeurs interdites, de même que le « h » représente le « h » de hachurage, ainsi il est naturel de mettre ou non les barres au besoin.

Pour étendre le hachurage sur plus d'un des intervalles du domaine, il vous suffit de sauter l'élément suivant avec toujours la même notation, à savoir `()`

Example :

```
  #tabvar(
    variable: $t$,
    label: (
      ([Pour `|h|`], "s"),
      ([Pour `h|`], "s"),
      ([Pour `|h`], "s"),
      ([Pour `h`], "s"),
    ),
    domain: ($ 2 $, $4$, $5$, $8$,
$9$),
    contents: (
      ($+$, "|h|", (), $-$),
      ($-$, "h|", $-$, $+$),
      ($+$, "|h", (), $-$),
      ($-$, "h", $-$, $+$),
    ),
  )
```



3.2 - Format pour les variations

Il doit être positioné au même index dans l'array `contents` qu'un label possédant le string `"v"`, ce qui indique que la ligne doit être considérée comme un tableau de variations.

De plus, il doit avoir autant d'éléments que le domaine, sinon le programme renvoie une erreur.

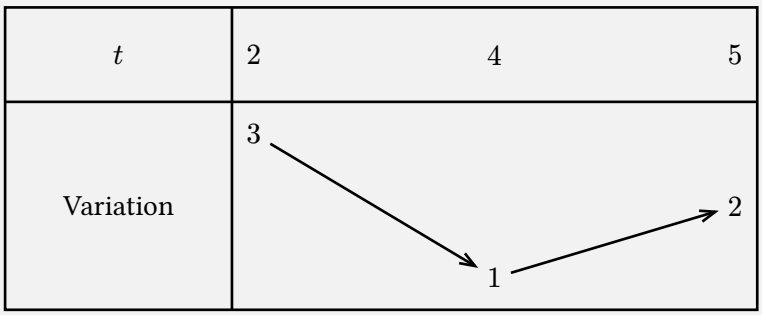
Chacun des éléments qui le compose doit être sous l'une de ces formes :

- `()` - Vide : pour étendre la flèche précédente au prochain élément
- `(position, body)` - Le cas classique, constitué de la position de l'élément (top, center, bottom), et du body
- `(pos1, pos2, "||", body1, body2)` - Le cas où l'élément est non défini
- `(pos, "||", body)` - Une écriture condensée de la forme précédente
- `"h"` ou `"|h"` - La balise de début d'une zone hachurée
- `"H"` ou `"H|"` - La balise de fin d'une zone hachurée

3.2.1 - Array classique pour les sous-tableaux de variations

Un array pour les sous-tableaux de variations doit contenir au moins deux éléments, à savoir la position et l'élément lui-même.
La position peut être l'un des trois éléments `top`, `center` ou `bottom`, mais ne peut être d'aucun autre type d'alignement.

```
#tabvar(
  variable: $t$,
  label: (
    ([Variation], "v"),
  ),
  domain: ($ 2 $, $4$, $5$),
  contents: (
    (
      (top, $3$),
      (bottom, $1$),
      (center, $2$),
    ),
  ),
)
```



3.2.2 - Valeurs interdites

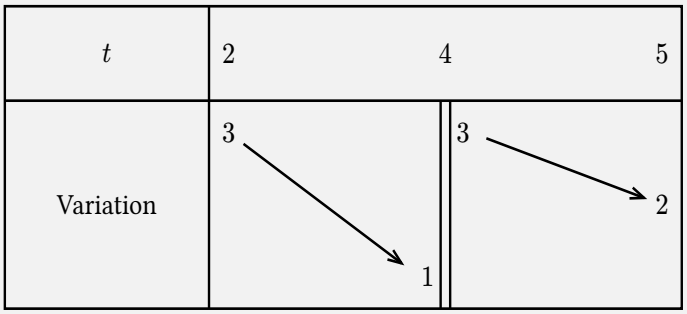
Si votre fonction n'est pas définie en certains points, comme $f(x) = \frac{1}{x}$ pour $x = 0$, vous voudrez sans doute mettre une double barre pour signifier que ces valeurs ne sont pas définies.

⋆ Pour chaque valeur du domaine excepté le début et la fin :

l'array doit avoir cette forme `(pos1, pos2, "||", élément1, élément2)`, où :
- `pos1` et `pos2` sont au choix `top`, `center` ou `bottom`
- `"||"` est là pour spécifier que la valeur est non définie
- `élément1` et `élément2` sont de type `contents` où `élément1` est avant la barre, et `élément2` après.

Exemple.

```
#tabvar(
  variable: $t$,
  label: (
    ([Variation], "v"),
  ),
  domain: ($ 2 $, $4$, $5$),
  contents: (
    (
      (top, $3$),
      (bottom, top, "||", $1$, $3$),
      (center, $2$),
    ),
  ),
)
```

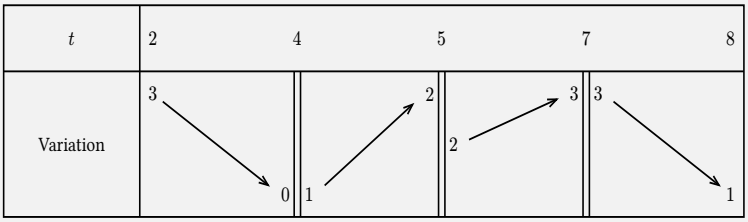


Dans le cas où `pos1` et `pos2` sont identiques, vous pouvez n'en mettre qu'un seul des deux, de même pour `élément1` et `élément2`

Exemple.

À la place de `(top, top, "||", $0$, $0$)`, vous pouvez mettre `(top, "||", $0$)`

```
#tabvar(
  variable: $t$,
  label: (
    ([Variation], "v"),
  ),
  domain: ($ 2 $, $4$, $5$, $7$, $8$),
  contents: (
    (
      (top, $3$),
      (bottom, "||", $0$, $1$),
      (top, center, "||", $2$),
      (top, "||", $3$),
      (bottom, $1$),
    ),
  ),
)
```

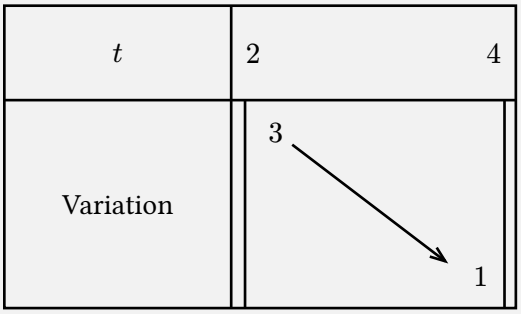


⋆ Pour le début et la fin

Ici comme il n'y a qu'un élément, l'array est comme la notation compressée vue précédement, ie. : `(pos, "||", élément)`

Exemple.

```
#tabvar(
  variable: $t$,
  label: (
    ([Variation], "v"),
  ),
  domain: ($ 2 $, $4$),
  contents: (
    (
      (top, $3$),
      (bottom, "||", $1$),
    ),
  ),
)
```

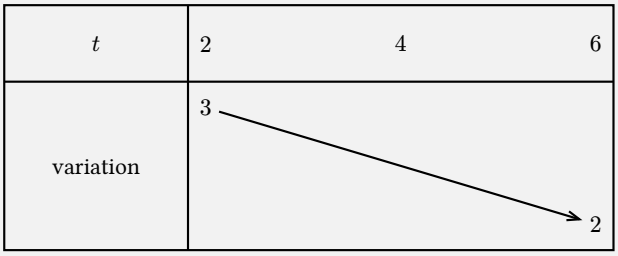


3.2.3 - Ignorer une valeur

Quand vous utilisez plusieurs fonctions dans un même tableau de variations, vous voudrez probablement ignorer certaines valeurs du domaine. Pour cela, comme pour les sous-tableaux de signes, il suffit de mettre un array vide « `()` »

Exemple.

```
#tabvar(
  variable: $t$,
  label: (([variation], "v"),),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $3$),
      (),
      (bottom, $2$),
    ),
  ),
)
```



3.3 - Hachurage pour un intervalle non défini

À la différence des sous-tableaux de signes, ici, les éléments portent sur chacune des valeurs du domaine, et non sur les intervalles.
Ainsi, pour indiquer que la fonction n'est pas définie sur un certain intervalle, on utilisera quatre balises, dont deux « d'ouverture » et deux de « fermeture ».

- Les balises « d'ouverture » sont : `"h"` et `"|h"`, la seconde balise précise que la fonction n'est pas définie pour cette valeur.
- Les balises de « fermeture » sont : `"H"` et `"H|"`, la seconde balise précise que la fonction n'est pas définie pour cette valeur.

Ainsi, il vous suffira de mettre cette balise entre l'alignement et la valeur de la fonction, par exemple `(top, [balise], $3$)`

De plus, si vous voulez étendre les hachures sur plus d'un intervalle, il vous suffit de mettre des array vidse entre les deux éléments contenant une balise d'ouverture et de fermeture.

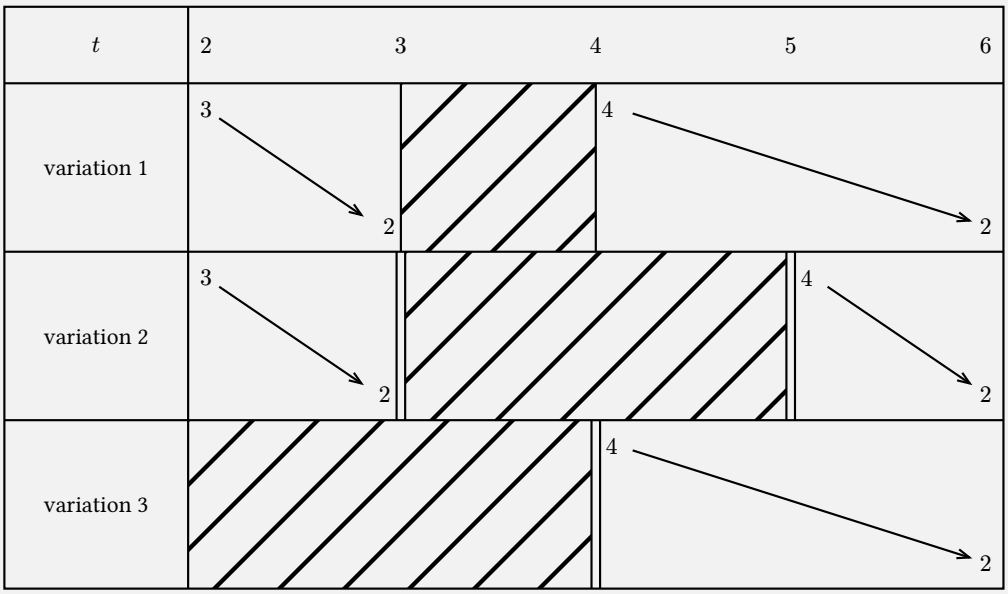
Attention.

- les balises « `|h` » et « `H|` », ne sont respectivement pas compatibles avec le premier élément et le dernier élément.

- faites attention à ne pas mettre d'élément non vide entre deux balises, ceci casse le tableau.

Exemple :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation 1], "v"),
    ([variation 2], "v"),
    ([variation 3], "v"),
  ),
  domain: ($2$, $3$, $4$, $5$, $6$),
  contents: (
    (
      (top, $3$),
      (bottom, "h", $2$),
      (top, "H", $4$),
      (),
      (bottom, $2$),
    ),
    (
      (top, $3$),
      (bottom, "|h", $2$),
      (),
      (top, "H|", $4$),
      (bottom, $2$),
    ),
    (
      (bottom, "|h", $2$),
      (),
      (top, "H|", $4$),
      (),
      (bottom, $2$),
    ),
  ),
)
```



4 - Redimensionner le tableau

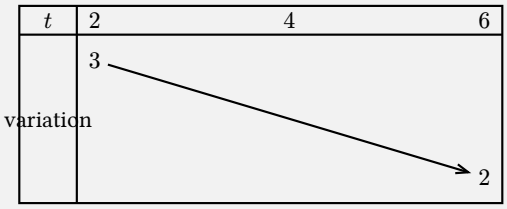
4.1 - Première ligne et première colonne

Comme indiqué en section 2, il existe deux paramètres effectuant exactement ce dont il est question ici, ie. modifier la première ligne et la première colonne.

Ces deux paramètres prennen un type `length`, ils doivent être toutefois positifs !

Exemple.

```
#tabvar(
  variable: $t$,
  label: (([variation], "v"),),
  domain: ($2$, $4$, $6$),
  first-column-width: 1cm,
  first-line-height: 5mm,
  contents: (
    (
      (top, $3$),
      (),
      (bottom, $2$),
    ),
  ),
)
```



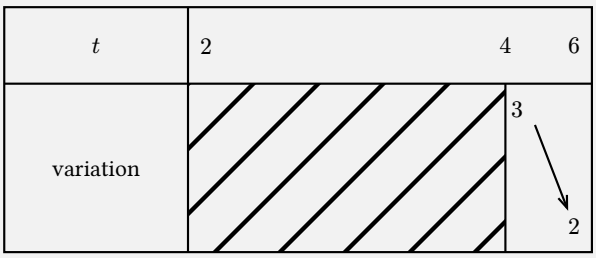
NB. Si ces deux paramètres ne sont pas remplis, alors la hauteur et la largeur se caleront sur la taille du texte contenu.
Cependant, si celui-ci est trop petit, alors la première colonne fera 30mm de largeur et la première ligne fera 12mm de haut.

4.2 - Redimensionner l'espacement entre les éléments

Pour modifier l'écart entre les éléments du domaine, remplacez l'élément avant l'écart à modifier par un couple de la forme « `(content, length)` », où `content` est l'élément du domaine à cet endroit, et `length` la distance entre cet élément et le prochain.
Ainsi comme vous l'avez compris, le dernier élément ne peut être remplacé par un tel couple.

Exemple.

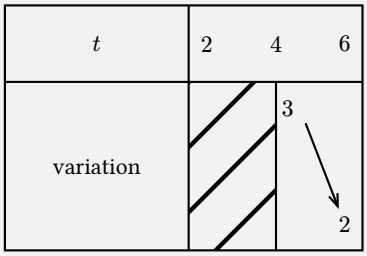
```
#tabvar(
  variable: $t$,
  label: (([variation], "v"),),
  domain: (($2$, 5cm), ($4$, 1cm), $6$),
  contents: (
    (
      "h",
      (top, "H", $3$),
      (bottom, $2$),
    ),
  ),
)
```



Si vous voulez modifier tous les écarts de la même manière, il vous suffit d'utiliser le paramètre `element-distance`

Example :

```
#tabvar(
  variable: $t$,
  label: (([variation], "v"),),
  domain: ($2$, $4$, $6$),
  element-distance: 1cm,
  contents: (
    (
      "h",
      (top, "H", $3$),
      (bottom, $2$),
    ),
  ),
)
```



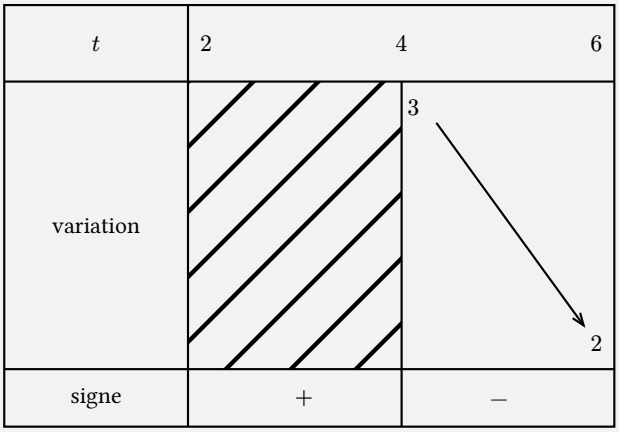
4.3 - Redimensionner la hauteur des sous-tableaux

Pour modifier cette hauteur, rajoutez dans le label, dans l'array correspondant au sous-tableau, entre le content et la balise « signes » `"s"` ou « variations » `"v"`, la hauteur que vous souhaitez.

Par défaut, cette hauteur est au minimum de 13,5 mm

Example :

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], 5cm, "v"),
    ([signe], 10mm, "s"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      "h",
      (top, "H", $3$),
      (bottom, $2$),
    ),
    ($+$, $-$),
  ),
)
```



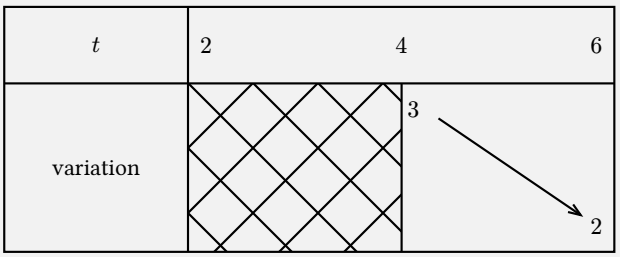
5 - Customisation du hachurage

Pour cela, il suffit de mettre un objet de type `tiling` au paramètre `hatching-style`

Exemple.

```
  #tabvar(
    #let exemple = tiling(size: (30pt, 30pt))[
    #place(line(start: (0%, 0%), end: (100%,
100%)))
    #place(line(start: (0%, 100%), end: (100%,
0%)))
  ]

  #tabvar(
    variable: $t$,
    label: (
      ([variation], "v"),
    ),
    domain: ($2$, $4$, $6$),
    hatching-style: exemple,
    contents: (
      (
        "h",
        (top, "H", $3$),
        (bottom, $2$),
      ),
    ),
  )
```



De plus, le paquet vient avec sont lot de hachures prédéfinies par Alkion (merci à lui), dont en voici la présentation :

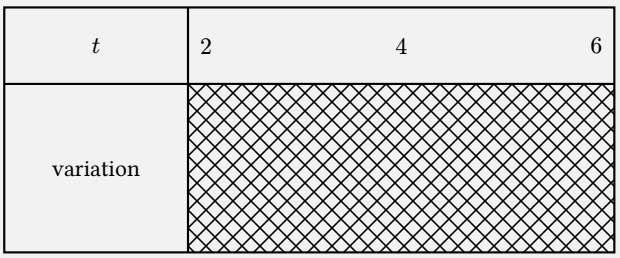
⋆ `grille`

Définition :

```
  #let grille = tiling(size: (8pt, 8pt))[
    #place(line(start: (0%, 0%), end: (100%, 100%), stroke: .7pt))
    #place(line(start: (0%, 100%), end: (100%, 0%), stroke: .7pt))
  ]
```

Exemple.

```
  #tabvar(
    variable: $t$,
    label: (
      ([variation], "v"),
    ),
    domain: ($2$, $4$, $6$),
    hatching-style: grille,
    contents: (
      (
        "h",
        (),
        (top, "H", $3$),
      ),
    ),
  )
```



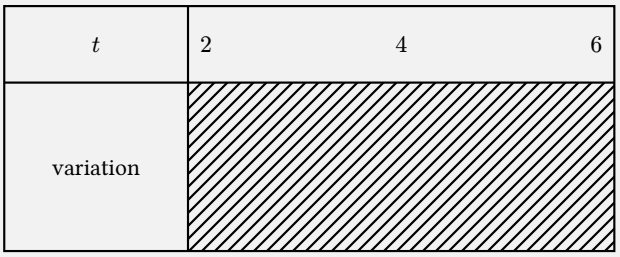
⋆ `nelines`

Définition :

```
#let nelines = tiling(size: (6pt, 6pt))[
  #place(line(start: (100%, 0%), end: (0%, 100%)))
  #place(line(start: (100%, -100%), end: (-100%, 100%)))
  #place(line(start: (200%, 0%), end: (0%, 200%)))
]
```

Exemple.

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
  ),
  domain: ($2$, $4$, $6$),
  hatching-style: nelines,
  contents: (
    (
      "h",
      (),
      (top, "H", $3$),
    ),
  ),
)
```



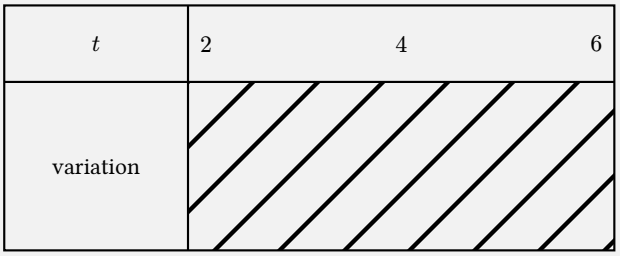
⋆ `bignelines`

Définition :

```
#let bignelines = tiling(size: (30pt, 30pt))[
  #place(line(start: (100%, 0%), end: (0%, 100%), stroke: 2pt))
  #place(line(start: (100%, -100%), end: (-100%, 100%), stroke: 2pt))
  #place(line(start: (200%, 0%), end: (0%, 200%), stroke: 2pt))
]
```

Exemple.

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
  ),
  domain: ($2$, $4$, $6$),
  hatching-style: bignelines,
  contents: (
    (
      "h",
      (),
      (top, "H", $3$),
    ),
  ),
)
```



⋆ `nwlines`

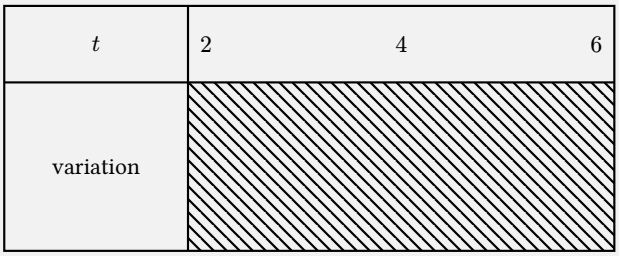
Définition :

```
#let nwlines = tiling(size: (6pt, 6pt))[
  #place(line(start: (0%, 0%), angle: 45deg))
  #place(line(start: (-100%, 0%), angle: 45deg))
```

```
    #place(line(start: (100%, 0%), end: (200%, 100%)))
    #place(line(start: (100%, -100%), angle: -135deg))
  ]
```

Exemple :

```
  #tabvar(
    variable: $t$,
    label: (
      ([variation], "v"),
    ),
    domain: ($2$, $4$, $6$),
    hatching-style: nwlines,
    contents: (
      (
        "h",
        (),
        (top, "H", $3$),
      ),
    ),
  )
```



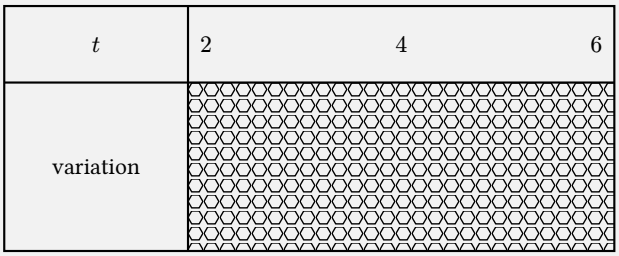
⋆ hatch

Définition :

```
  #let hatch = tiling(size: (7pt, 7pt))[
    #place(polygon.regular(vertices: 6, size: 6.5pt, stroke: .6pt))
  ]
```

Exemple.

```
  #tabvar(
    variable: $t$,
    label: (
      ([variation], "v"),
    ),
    domain: ($2$, $4$, $6$),
    hatching-style: hatch,
    contents: (
      (
        "h",
        (),
        (top, "H", $3$),
      ),
    ),
  )
```



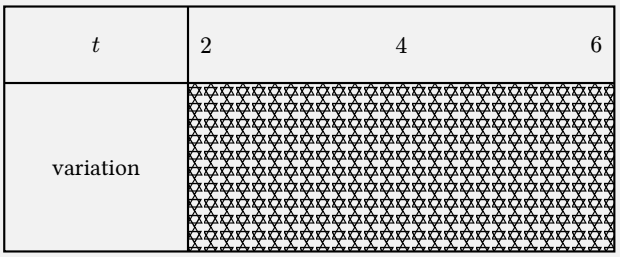
⋆ etoile

Définition :

```
  #let etoile = tiling(size: (7pt, 7pt))[
    #place(rotate(180deg, origin: center + horizon)[#polygon.regular(vertices: 3,
size: 6.5pt, stroke: .6pt)])
    #place(polygon.regular(vertices: 3, size: 7pt, stroke: .5pt))
  ]
```

Exemple.

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
  ),
  domain: ($2$, $4$, $6$),
  hatching-style: etoile,
  contents: (
    (
      "h",
      (),
      (top, "H", $3$),
    ),
  ),
)
```

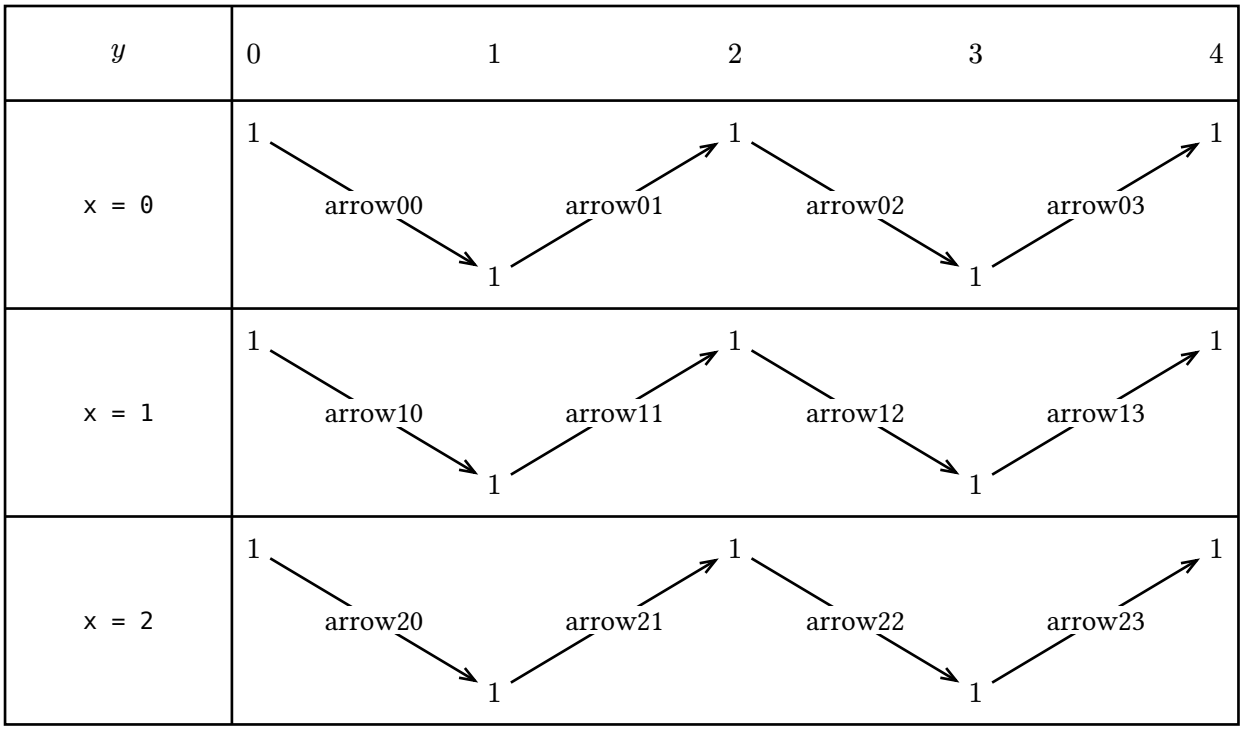


6 - Ajouter des valeurs dans les sous-tableaux de variations

Il est en effet possible d'ajouter des valeurs dans les sous-tableaux de variations, sans allonger le domaine.
Ce cas peut être utile à ceux qui voudraient expliciter sur leur tableaux une valeur précise prise par votre fonction, dû au théorème des valeurs intermédiaires.

C'est ici que l'argument `values` sert. En effet vous allez mettre dans `values` autant de valeurs que vous voulez ajouter.
Les éléments que vous ajoutez doivent avoir cette forme : `("arrowxy", content1, content2)`, où :

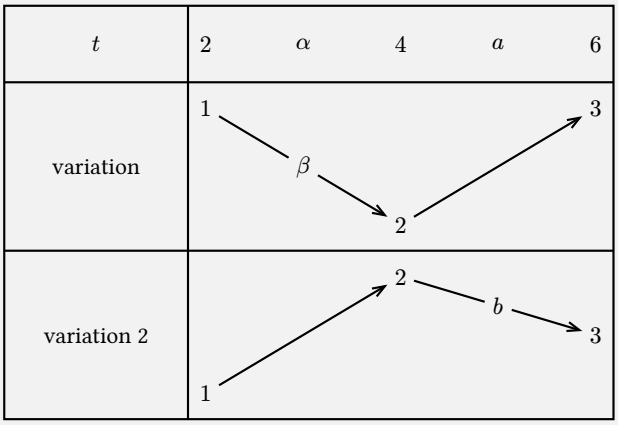
- `x` et `y` dans `"arrowxy` sont les coordonnées de la flèche sur laquelle vous voulez ajouter une valeur, ces coordonées commencent en haut à gauche par `x = 0, y = 0`



- `content1` le content qui sera placé au niveau du domain
- `content2` le content qui sera placé sur la flèche

Example :

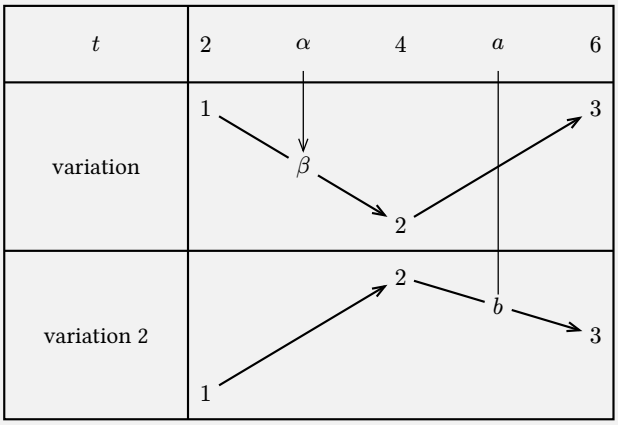
```
#tabvar(
  fill-color: luma(95%),
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  values: (
    ("arrow00", $alpha$, $beta$),
    ("arrow11", $a$, $b$),
  ),
)
```



De plus, il est possible d'ajouter une flèche ou une ligne reliant la valeur dans le domaine et celle sur la flèche.

Exemple.

```
#tabvar(
  fill-color: luma(95%),
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  values: (
    ("arrow00", $alpha$, $beta$, "f"),
    ("arrow11", $a$, $b$, "l"),
  ),
)
```



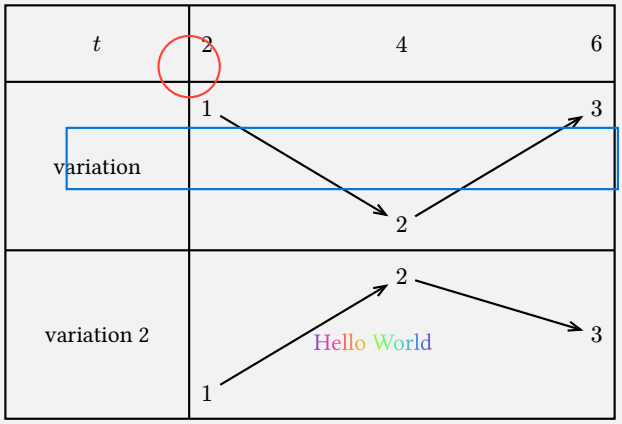
7 - Ajouter ce que vous voulez avec `add`

Il est en effet possible d'ajouter autant d'éléments que vous voulez (tant que Cetz le peut) à vos tableaux, pour cela il suffit d'ajouter ces éléments dans le paramètre `add`.

Attention. Pour ajouter des éléments propres à Cetz, comme `content`, `rect`, etc. vous devez les importer dans votre fichier Cetz.

Exemple.

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  add: {
    cetz.draw.circle((3, -1), stroke: red, radius: 5mm)
    cetz.draw.content((6, -5.5), text(fill: gradient.linear(..color.map.rainbow))
[Hello World])
    cetz.draw.rect((1, -2), (10, -3), stroke: blue)
  },
)
```



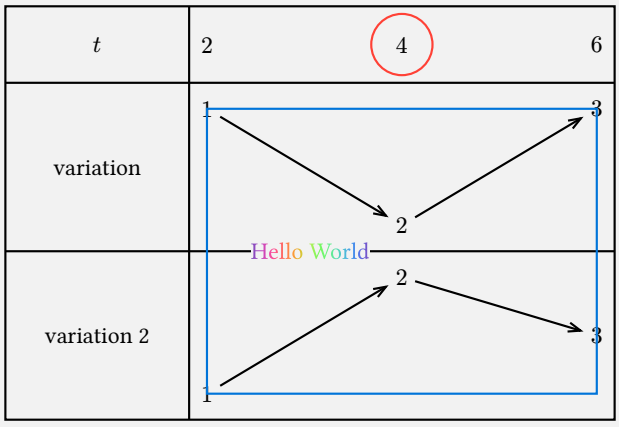
Pour simplifier le processus, chaque élément du tableau possède un « nom » qui permet, par le système de coordonnées de Cetz, d'attacher les éléments ajoutés aux éléments déjà présents.
Voici un tableau qui résume les noms :

Éléments	nom	précision
Variable	`var`	
Domaine	`domainx`	x représente le x-ième élément du domaine
Label	`labely`	y représente le y-ième label
Ligne entre les sous-tableaux	`line-betwen-table-nby`	y représente la y-ième ligne Note : la ligne 0 sépare le domaine du reste
Cadre	`cadre`	toujours utilisable, même avec `nocadre` à `true`
Ligne entre les labels et les sous-tableaux	`line-separating-labels-tables`	
Ligne passant au niveau du domaine, centré	`line-centred-domain`	cette ligne n'est pas visible

Flèches dans les sous-tableaux de variation	`arrowxy`	exactement ceux rencontrées dans la section 6 S'y référer pour plus de précision
Éléments dans un sous-tableau de variations	`variationxy`	x et y sont les coordonnées de l'élément
Éléments dans un sous-tableau de signes	`signxy`	x et y sont les coordonnées de la barre. Fonctionne de la même manière que `arrowxy`.
Hachurage	`hatchingxy`	x et y sont les coordonnées du hachurage
Élément dans le domaine, pour une valeur ajoutée	`depart_valuesx`	x est le x-ième élément ajouté
Élément dans le sous-tableau de variations, pour une valeur ajoutée	`fin_valuesx`	x est le x-ième élément ajouté

Exemple 1.

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  add: {
    import cetz.draw: *
    circle("domain1", stroke: red, radius: 5mm)
    content(
      "line-betwen-table-nb1",
      text(fill: gradient.linear(..color.map.rainbow))[Hello World],
      frame: "rect",
      fill: luma(95%),
      stroke: none,
    )
    rect("variation02", "variation10", stroke: blue)
  },
)
```



Exemple 2.

```
#tabvar(
  variable: $t$,
  label: (
    ([variation], "v"),
    ([variation 2], "v"),
  ),
  domain: ($2$, $4$, $6$),
  contents: (
    (
      (top, $1$),
      (bottom, $2$),
      (top, $3$),
    ),
    (
      (bottom, $1$),
      (top, $2$),
      (center, $3$),
    ),
  ),
  add: {
    import cetz.draw: *
    polygon("line-centred-domain.82%", 5, stroke: red, radius: 5mm)
    line("var", "label1", stroke: blue)
    image("")
  },
)
```

